

NLP & Visión Computacional | Tarea 03 - Filtros Aplicados a Imágenes

Aguirre Calzadilla César Miguel

Resumen

Se presenta una comparativa de rendimiento entre la ejecución de filtros de convolución de imágenes en CPU y GPU mediante el uso de CUDA. Implementamos tres filtros: Filtro de Media, Magnitud del Gradiente y Filtro Laplaciano, aplicándolos a la obra "La Ofrenda" de Saturnino Herrán. Los resultados demuestran un speedup abrumador a favor de la GPU. El Filtro de Media, alcanzó uno de 30x, mientras que el Gradiente y Laplaciano mostraron aceleraciones de 6x y 2x.

Keywords: CPU, GPU, Convolución, Filtro de Media, Magnitud del Gradiente, Filtro Laplaciano, Speedup.

Correo electrónico: cesar.aguirre@cimat.mx

1. Introduction

Dentro del procesamiento de imágenes y la visión computacional, hay una herramienta matemática que destacó de entre muchas: la convolución. Esta operación se puede describir como una ventana por la cual observar, llamada kernel, que se desliza por cada pixel de la imagen con la que estemos trabajando. En cada posición, realiza una operación matricial entre los valores del kernel y los pixeles que cubre. El resultado define los nuevos valores de los pixeles en una tamaño igual al del kernel. A este proceso se le conoce como "filtrado", y es parte fundamental de lo que ahora conocemos como Convolutional Neural Networks (CNN) en Deep Learning.

Entre los filtros más populares están: el Filtro de Media, la Magnitud del Gradiente de la Imagen, y el Filtro Laplaciano.

1.1. Filtro de Media

Este filtro es un operador de suavizado que intenta atenuar el ruido dentro de una imagen, así como las variaciones de intensidad en una vecindad definida por el kernel. Su resultado suele ser un desenfoque controlado de la representación que estemos trabajando. Su representación matricial es la siguiente

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Para normalizar y obtener un promedio, la suma de los productos se divide entre la suma de los elementos del kernel, que en este ejemplo son nueve.

1.2. Magnitud del Gradiente de la Imagen

Este método se utiliza principalmente para detectar discontinuidades en la imagen, las cuales corresponden con bordes de los objetos dentro de la representación. El gradiente de la imagen se aproxima mediante diferencias finitas a lo largo de los ejes X y Y. La magnitud del gradiente cuantifica la tasa de cambio de intensidad en cada pixel, un valor elevado indica la presencia de un borde.

Las derivadas parciales y discretas y la magnitud del gradiente se calculan de la siguiente manera:

- **Derivada del eje X:** $D_x(x, y) = I(x, y) - I(x - 1, y)$
- **Derivada del eje Y:** $D_y(x, y) = I(x, y) - I(x, y - 1)$
- **Magnitud del Gradiente:** $GM(x, y) = \sqrt{D_x^2(x, y) + D_y^2(x, y)}$

Este filtro es sumamente útil para representación de características.

1.3. Filtro Laplaciano

El operador Laplaciano es un filtro de detección de bordes que se basa en la segunda derivada de la información en las imágenes. Su mecanismo es sensible a las discontinuidades, esto es de particular interés pues a diferencia de otros métodos basados en gradientes, el

Laplaciano puede identificar si un pixel pertenece a un lado oscuro o claro de un borde.

Su representación matricial es la siguiente:

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Pese a que el estándar actual para la identificación de objetos y clasificación de imágenes sean las redes neuronales, muy en particular las Redes Neuronales Convolucionales (CNN), los kernel optimizados en las capas iniciales frecuentemente convergen en formas que se asemejan a operadores fundamentales como lo es el Laplaciano. Los principios matemáticos y la aplicación de filtros como los que hemos visto ahora son la base para comprender de mejor manera cómo funcionan las CNN y cómo se ha ido construyendo la teoría alrededor de la Visión Computacional.

2. Implementación

```
1 import numpy as np
2 from scipy.ndimage import convolve
3
4 def apply_essential_filters(imagen,
5                             filter_sequence):
6     def aplicar_filtro_media(img):
7         kernel = np.ones((5, 5)) / 25
8         return convolve(img, kernel)
9
10    def aplicar_filtro_laplaciano(img):
11        kernel = np.array([[0, 1, 0], [1, -4,
12        1], [0, 1, 0]])
13        return convolve(img, kernel)
14
15    def aplicar_magnitud_gradiente(img):
16        dx = np.diff(img, axis=1, append=0)
17        dy = np.diff(img, axis=0, append=0)
18        return np.sqrt(dx**2 + dy**2)
19
20    filtros = {
21        'media': aplicar_filtro_media,
22        'gradiente': aplicar_magnitud_gradiente,
23        'laplaciano': aplicar_filtro_laplaciano
24    }
25    resultados = {}
26    for nombre_filtro in filter_sequence:
27        if nombre_filtro in filtros:
28            resultados[nombre_filtro] = filtros[
29            nombre_filtro](imagen)
30
31    return resultados
```

Código 1. Código aplicación de filtros para CPU

Esta sección del código define los kernels de convolución y el bucle que los aplica a la imagen. Este fragmento combina la lógica seguida

para la función de `apply_filters_cpu` y las definiciones de los filtros. Se define un diccionario que asocia los nombres de los filtros con su función correspondiente, cada función de filtro define un kernel o realiza un cálculo equivalente como el gradiente, y el bucle itera sobre una secuencia de filtros definida por el usuario. Se aplica cada filtro invocando a la imagen original mediante la función `convolve`.

```
1 import cupy as cp
2 import numpy as np
3
4 def apply_laplaciano_gpu_essential(imagen_cpu):
5     h, w = imagen_cpu.shape
6
7     img_gpu = cp.asarray(imagen_cpu)
8     out_gpu = cp.zeros_like(img_gpu)
9
10    block = (16, 16) # 256 hilos por bloque
11    grid = ((w + block[0] - 1) // block[0], (h +
12        block[1] - 1) // block[1])
13
14    laplaciano_kernel(grid, block, (img_gpu,
15        out_gpu, w, h))
16
17    return cp.asnumpy(out_gpu)
```

Código 2. Código aplicación de filtros para GPU

Esta es la versión GPU con CUDA. La implementación se centra en la definición de kernel de CUDA, i.e el código C++ que se ejecuta en la GPU, y la función de python que lo invoca, manejando la transferencia de memoria entre la CPU y la GPU. El código paralelo ejecuta los hilos en la GPU y el código host en python prepara los datos para lanzarlos. Se condensa en 2 el ciclo completo, primero definimos el kernel CUDA, se transfiere la imagen de la memoria principal a la VRAM, configuramos los hilos de GPU, lanzamos el kernel y recuperamos el resultado desde la GPU hacia la CPU.

3. Resultados

3.1. Aplicación en CPU



Figura 1. Filtros aplicados en CPU para 'La Ofrenda'.

En la figura 1 se nos muestra el resultado de aplicar nuestros tres filtros a 'La Ofrenda' (1913), pintura de Saturnino Herrán¹. Podemos

¹ Esta pintura fue realizada por un pintor de aguascalientes, y es una de las obras más populares y emblemáticas del arte mexicano. Fue creada en medio de la Revolución Mexicana, con pintura al óleo que va más allá de lo que usualmente se acostumbraba. Se convirtió, o la convirtieron, en un símbolo de identidad nacional en un momento de crisis y transformación. Esta obra nos sitúa en los canales de Xochimilco, donde una familia indígena viaja en trajinera para llevar su ofrenda de Día de los Muertos. Vemos que hay tres figuras evidentes: un bebé, una joven y un anciano representando el ciclo de la vida y muerte.

notar que para el filtro de medias se realizó un desenfoque bastante potente, esto debido al kernel de 20 que le fijamos. Asimismo, podemos ver que para Gradiente y Laplaciano destacan los bordes de cada estructura y figura de la pintura, se resaltan los cambios en cada canal RGB, como los intensos naranjas (rojos) de la flor de cempasúchil o los azules en la trajinera. Es notorio que esta pintura tiene una fuerte textura, pues los filtros Laplaciano y Gradiente detectan muchos bordes. Diría que en este caso, Laplaciano es un filtro que detecta cambios más finos, mientras que Gradiente es un poco más difuminado todo.

3.2. Aplicación en GPU



Figura 2. Filtros aplicados en GPU para 'La Ofrenda'.

La figura 2 representa los resultados obtenidos al aplicar los filtros en GPU. En general, hay bastante consistencia en cuanto a lo que se obtuvo. Cada filtro mantiene características muy similares, pero quizás lo que salta un poco a la vista es que en los filtros de detección de bordes, la aplicación en GPU logra encontrar bordes más intensos en los canales RGB, que no se aprecian en CPU. Esto no estoy seguro de que sea por un poder de procesamiento mejor, creo que va más por el lado de que las librerías de CPU y GPU son distintas, entonces cada una interpreta la información de maneras diferentes, pero con una consistencia general clara.

3.3. Comparativa en tiempos de ejecución

La figura 3 muestra los tiempos de ejecución en CPU contra los tiempos de ejecución en GPU. El tiempo de ejecución en GPU es abrumadoramente más rápido que el tiempo de ejecución en CPU.

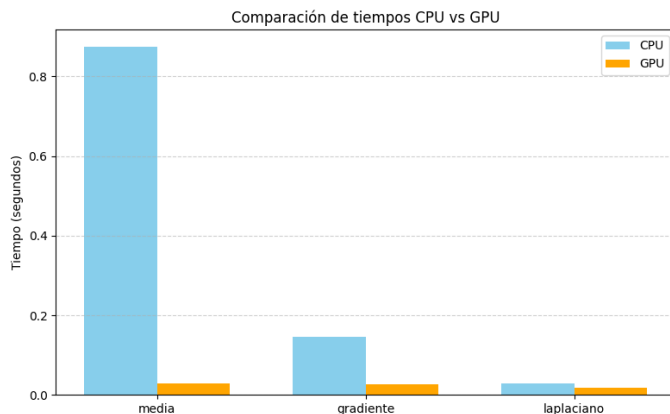


Figura 3. Tiempos de ejecución CPU vs. GPU para 'La Ofrenda'

Para complementar este radical impulso en tiempos de ejecución, podemos ver el speedup que se generó para cada filtro al observar los resultados obtenidos en la figura 4.

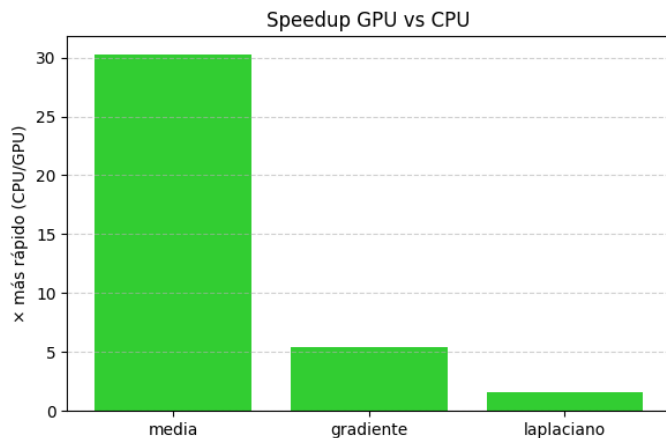


Figura 4. Speedup 'La Ofrenda'.

4. Conclusiones

La figura 4 es el resultado más importante para esta práctica. Si bien podemos entrar en detalle respecto a las diferencias en cuanto a las imágenes obtenidas por cada filtro tanto en CPU como en GPU, el objetivo principal de la tarea era demostrar cómo con métodos de paralelización en tarjetas gráficas podíamos minimizar considerablemente los tiempos de ejecución de estas operaciones, al compararse con los tiempos de trabajo en CPU. En ese sentido, la figura 4 nos muestra que, para el filtro de Media, el speedup fue 30 veces más veloz en GPU, el filtro de Gradiente fue alrededor de seis veces más rápido y el Laplaciano fue aproximadamente dos veces más veloz.

En nuestro caso, jugamos con un kernel de 20 para el filtro de Medias. Tener un filtro de medias con un kernel tan grande es esencial para comprender el porqué de un speedup tan evidente para este caso particular. La diferencia más grande es la cantidad de trabajo que se necesita para calcular el valor de un solo píxel de salida. Para cada uno de nuestros píxeles, necesitamos acceder a una ventana de $20 \times 20 = 400$ píxeles. Es decir, tenemos que sumar esos 400 píxeles y luego dividirlos entre 400, lo cual resulta en una cantidad enorme de operaciones por cada píxel, a nivel comparativa entre CPU y GPU al menos.

Contrastando con el filtro de Media, tenemos el de Gradiente y el Laplaciano. Estos filtros solo se apoyan de una ventana de 3×3 . Esto significa que para cada píxel de salida, solo necesitamos acceder a nueve vecinos y realizar las operaciones pertinentes.

Entonces, la alta paralelización de las GPUs permiten que el proceso sea mucho más rápido en el caso particular del filtro de Media. Esto no significa que para Laplaciano y Gradiente el resultado sea despreciable, reducir los tiempos de cómputo de dos a seis veces demuestra ya un salto en eficiencia muy significativo.

Así, podemos entonces comprender cómo es que la GPU está diseñada para el trabajo repetitivo y pesado. El filtro de medias con un kernel grande representa justo eso, una operación simple como lo es sumar y dividir, pero repetido cientos de veces para cada píxel. La GPU puede asignar múltiples de sus núcleos para que cada uno trabaje con un píxel distinto, simultáneamente, procesando esas 400 sumas en paralelo a lo largo de toda la imagen. El resultado es, entonces, una aceleración sorprendente.

5. Apéndice

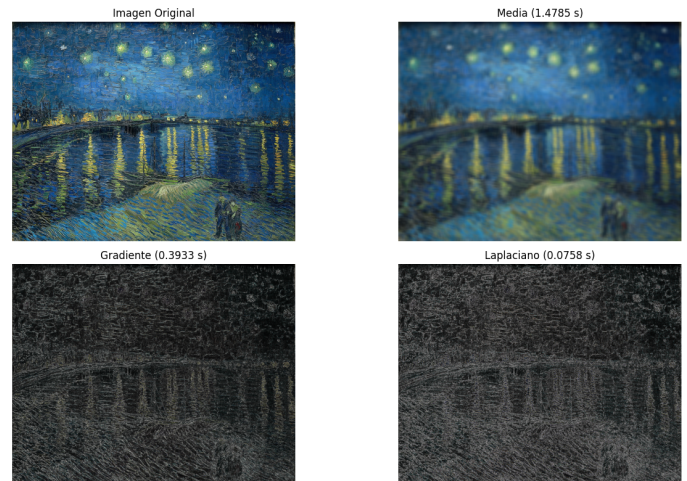


Figura 5. Noche Estrellada Sobre Ródano (1888) | Van Gogh.



Figura 6. Dánae (1636) | Rembrandt.

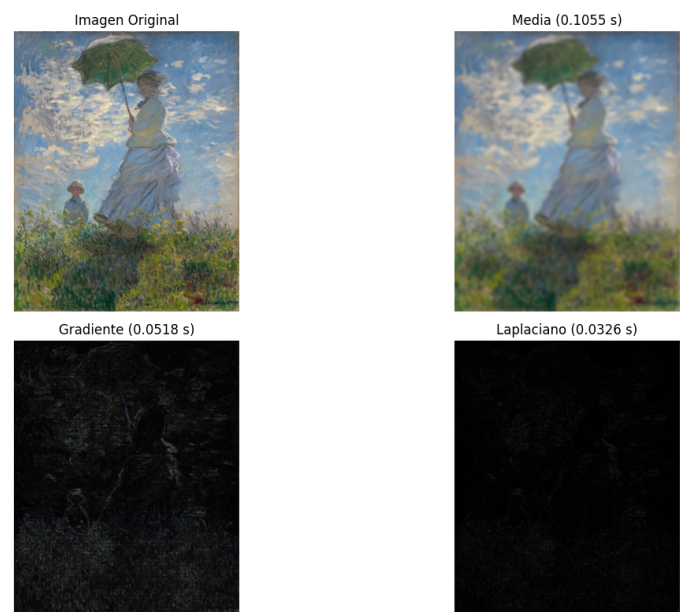


Figura 7. Mujer con Sombrilla (1875) | Claude Monet.